NATIONAL UNIVERSITY OF OSTROH ACADEMY
Department of Information Technologies and Data Analytics

Lab 12-13. TypeScript in practice and AI-assisted migration: typing API responses, union types, narrowing and type guards, any, type assertions, strict mode, and runtime validation

Modern Web Development: HTML, CSS, JS/TS

Deadline: 22.10.2026

Lab 12 and Lab 13 are combined into a single lab — Lab 12-13: both work with the same application.

1. Aim

Learn how to type a frontend application in TypeScript: describe API responses with types based on documentation, type functions, narrow types, and validate network data at runtime. Learn how to migrate JavaScript code to TypeScript with an AI agent and assess the quality of its output: `any`, type assertions, strict mode, and API response validation.

Expected outcome: the ShowFinder application runs entirely in TypeScript with the real TVmaze API; `npm run check` reports "Types: no errors" and all tests pass; you have written the types, requests, and states by hand, and you have verified and fixed the migration of the remaining modules performed by the AI agent.

2. Objectives and learning outcomes

After completing this lab, you must be able to:

- Describe API responses with TypeScript interfaces based on documentation and real responses, including `null` wherever it can appear.

- Type functions and classes; distinguish between `unknown` and `any`; narrow types (`typeof`, `instanceof`, `in`, checking discriminant properties) and write type guards.
- Model UI states using a discriminated union and handle them exhaustively with `never`.
- Read compiler errors and explain how TypeScript infers types (type inference).
- Explain what `tsc` does, what `strict` enables, and why types disappear after compilation.
- Explain what type assertions (`as`, `!`, `querySelector<T>()`) check and what they do not check, and replace them with narrowing.
- Validate network data at runtime and explain why a type definition alone is not enough.
- Migrate JavaScript code to TypeScript incrementally (`allowJs`) with an AI agent and verify the migration with the compiler, tests, and in the browser: what the agent fixed versus what it merely hid.

3. Lab tasks

1. Carry out the research tasks (§4): E1 in the browser, E2–E6 in the project as you reach each corresponding task.
2. Download the starter project, commit it to Git without changes, and run `npm run check` (§5.1).
3. Complete the assignment tasks (§4) in order: T1–T3 by hand, T4 as a migration with an AI agent (§5.2).
4. Complete the in-browser checklist (assignment, §6) and write the report (§6). Then proceed to the oral defense (§7).

4. Research tasks

Reference: Everyday Types (specifically Type Assertions and Non-null Assertion Operator), Narrowing, Object Types, Migrating from JavaScript — TypeScript Handbook; strict — TSConfig Reference; TVmaze API documentation.

Experiment protocol:

1. Read the description and code, then record what you expect — this is "My prediction".
2. Run the experiment.
3. If your prediction did not match the result, explain where it was wrong.

Summarize your findings in a table — it is part of your report:

#	What was tested	My prediction	Actual result	Explanation (if different)
E1	search API response	...	...	...

4.1. API response in the browser

E1. API response (before T1). In a new Chrome or Edge tab, open `https://api.tvmaze.com/search/shows?q=girls`, then the same URL with `?q=kitchen`. The browser displays JSON; enable **Pretty-print** to make it easy to read.

Analyze: what is at the top level of the response — an object or an array? What fields does each element consist of, and where is the show's title located? Which show from the `kitchen` query lacks a poster or a premiere date — and how does this look in JSON: is the property missing, or is it `null`? What can be `null`: `rating` itself or `rating.average`? Which of these must be described by the `Show` type (assignment, T1.2)?

4.2. TypeScript in the project

E2. Strict mode (after T1). Create a temporary file `src/strict.ts`:

```
import type { Show } from './types.ts';

export function posterUrl(show: Show) {
  return show.image.medium;
}

export function shout(text) {
  return text.toUpperCase();
}
```

Run `npm run typecheck`. Then, in `tsconfig.json`, temporarily replace `"strict": true` with `"strict": false` and run it again.

Analyze: what errors does the compiler show in `src/strict.ts` in each case? What would `posterUrl` do for the show Harlem's Kitchen (id 45962) at runtime? Without `strict`, a new error appears in `check/types.check.ts`: which rule of T1 cannot be checked without `strict`, and why? Restore `"strict": true` and delete `src/strict.ts`.

E3. unknown, any, and narrowing (after T2). In your editor, hover over `response.json()` in `getJson`, then in `searchShows` over the data returned by `getJson`: immediately after `await` and where you have already verified it with the guard. Temporarily add `console.log(data.length);` immediately after `await` (using your variable name), observe the error, and remove the line.

Analyze: what type does each of the three expressions have? Why is `getJson` declared as `Promise<unknown>` rather than `Promise<any>`? What would change in `searchShows` if it returned `any`?

E4. What remains after compilation (after T2). While `npm run watch` is running, open `dist/types.js` and `dist/api.js`.

Analyze: what remains of `interface Show`, `value is Show`, and `readonly status: number`? Why can't `isShow` be written as `value instanceof Show`? Why validate the shape

of the response at runtime (assignment, T2.7) if types already exist?

E5. Type assertions versus verification (after T2). Create a temporary file `src/assert.ts`:

```
import type { Show } from './types.ts';
import { isShow } from './api.ts';

const data: unknown = JSON.parse('{ "id": 1, "name": "Pilot" }');

if (isShow(data)) console.log('guard:', data.genres.join(', '));
else console.log('guard: not a show');

const show = data as Show;
console.log('as:', show.genres.join(', '));
```

Run `npm run typecheck`, then execute the file: `node src/assert.ts`. Next, append `const id = 'abc' as number;` at the end and run `npm run typecheck` again. Replace `as number` with `as unknown as number` and run it once more.

Analyze: did the compiler detect an error on the line with `as Show`? What does each `console.log` output, and where does the program crash? Why does TypeScript reject `'abc' as number`, but accept `'abc' as unknown as number`? What does `as` actually verify, and what does `isShow` verify? Delete `src/assert.ts`.

E6. DOM types (before T4). Create a temporary file `src/dom-types.ts`:

```
const byTag = document.querySelector('form');
const byId = document.querySelector('#search');
const claimed = document.querySelector<HTMLInputElement>('#search');
const asserted = document.querySelector('#search') as HTMLInputElement;

document.addEventListener('click', (event) => {
  const target = event.target;
});
```

Hover over each variable and over `target` in your editor. Then, on the application page in the Console, run `document.querySelector('#search') instanceof HTMLInputElement` and `document.querySelector('#search') instanceof HTMLFormElement`.

Analyze: what type does each variable have? How does the compiler know the type of `byTag`, but not `byId`? `claimed` and `asserted` promise an input element, but `#search` is a form: who verified that? Why is `event.target` typed as `EventTarget | null` rather than the element that was clicked? How can you obtain an `HTMLFormElement` without asserting anything? Delete `src/dom-types.ts`.

5. Practical part: how to work

The **ShowFinder** starter project is a TV series search app using TVmaze, written in JavaScript. The markup, styles, and the `app`, `view`, `dom`, and `main` modules are already provided — in JavaScript. Three TypeScript files are initially empty: `src/types.ts`, `src/api.ts`, and `src/state.ts`.

All practical tasks are in the assignment, §4: T1–T4, each with numbered rules, and in T4 — the exact prompt for the AI agent. Keep the assignment open while you work. This section explains the workflow: preparation, rules for working with the AI agent, and commits.

How to verify. `npm run check` first checks types (`tsc`), then runs tests (`node --test`), and finishes with a one-line summary: "Types: … · Tests: passed N of M". The name of each test and the comment next to each type check in `check/types.check.ts` begin with the rule number from the assignment. The tests make no network requests: they mock `fetch` and respond with recorded TVmaze fixtures, so they do not depend on Wi-Fi or rate limits. While the T4 modules remain in JavaScript, the T4 tests execute them directly: several of these tests fail even before migration because the existing JavaScript code contains defects. Testing against the real TVmaze API and browser is covered by the assignment's in-browser checklist (§6).

5.1. Preparation

1. Check `node -v`: Node.js 22.18 or newer is required. On older versions, the tests report `Unknown file extension ".ts"`.
2. Download showfinder-ts-starter.zip and extract it into a **separate repository** or into the `lab-12/` folder of your shared lab repository. Make the first commit **without changes**: `Starter`.
3. Run `npm install`, then `npm run watch` — and leave it running in a separate terminal.
4. Open `index.html` using Live Preview in Chrome or Edge: the search form is visible, and the Console shows `Uncaught Error: Not implemented`. This is expected: `app.js` calls functions from `src/state.ts` and `src/api.ts` that have not been implemented yet. The page will work after T3.
5. Run `npm run check`: initially there are many type errors and "Tests: passed 8 of 48". Read the starter project's `README.md`.
6. You only edit `src/types.ts`, `src/api.ts`, and `src/state.ts` by hand, and in T4 — the modules migrated by the agent. All other files, including `check/` and `tsconfig.json`, must remain identical to the starter project after every task.

5.2. Rules for working with an AI agent

- **Research (§4).** Write "My prediction" yourself, without an AI agent and before running

the experiment.

- **T1–T3 by hand.** The AI agent may explain TVmaze documentation, compiler errors, tests, or TypeScript concepts, but does not write code in `src/`. Example prompt in read-only mode (ask / plan):

  > Explain this TypeScript compiler error. Below are the error message, the line from check/types.check.ts with its rule comment, and my type definition. Do not write the corrected code: explain what is wrong with my type and where to read about it in the TypeScript Handbook.

- **Before T4.** After T3, `npm run check` reports "Types: no errors · Tests: passed 39 of 48". The remaining tests belong to T4: they fail due to defects in the JavaScript modules and because those modules have not yet been migrated to TypeScript. Run experiment E6. Do not fix the `.js` files yourself: that is part of T4.
- **T4 — four steps.**
  1. **AI migration.** Switch your IDE's AI agent to the mode where it edits project files directly (usually Agent). Close any editor tabs showing files from `check/`: many agents add open tabs to their context. Paste **only** the T4 prompt from the assignment into chat, word for word, with no extra clarifications of your own.
  2. **Migration commit.** Without fixing anything, commit **everything** the agent changed — even `tsconfig.json` or `check/` if it modified them: `T4: AI migration`. Commit even if the application does not work: this commit records what the agent produced.
  3. **Verification.** Run `npm run check`. For each type error or failed test, identify which T4 rule or §5 constraint was violated and why. Review the diff of the migration commit: every `as`, `!`, `any`, `declare`, `@ts-…`, and any change to `tsconfig.json` — what do they hide? Verify rule T4.7 in the browser using the assignment's checklist (§6).
  4. **Fixing by hand.** The AI may explain a test or an error, but **does not write** the fix. Revert any files the agent should not have touched using `git restore --source=HEAD~1 -- <file>`. Once `npm run check` passes with zero errors and the in-browser checklist succeeds, commit your fixes in a single commit: `T4: fixes`.

  If the migration passed all checks on the first attempt, omit the fixes commit and state in your report summary how you verified it.

In your report, specify the agent name and version, the model, and the date of work.

5.3. Commits

Git history for the lab:

1. `Starter` — starter project without changes;
2. `T1: types`;
3. `T2: api`;

4. `T3: state`;
5. `T4: AI migration` — the agent's migration without changes;
6. `T4: fixes` — if the migration did not pass verification.

You may commit more often, but each task concludes with a commit using this exact name. On GitHub, the `T4: fixes` commit shows exactly what you changed after the AI migration: during defense, you will use it to explain what was wrong with the agent's output. `node_modules/` and `dist/` are not committed, and you do not need to deploy the application: the instructor runs `npm run check` directly in your repository.

6. Report requirements

Email subject:

```
[Web] Lab 12-13: TypeScript and AI migration
```

In the email, include a link to the repository:

```
GitHub repository: https://github.com/<username>/<repository>
```

The report must contain:

1. the table of experiments from §4 (E1–E6);
2. a screenshot of `npm run check` showing the summary "Types: no errors · Tests: passed 48 of 48";
3. the in-browser checklist from the assignment (B1–B4) with screenshots;
4. the name and version of the AI agent, the model, and the date of work;
5. a summary of completed work: briefly describe what you wrote in T1–T3; what `npm run check` reported after the `T4: AI migration` commit and what was wrong with the migration (`any`, `as`, `!`, `declare`, `@ts-...`, modified configuration, overlooked defects); what you fixed.

7. Oral defense

The submission format includes an oral interview based on the practical implementation. The questions below are typical frontend developer interview questions covering the topics of this lab. Answer using examples from your own ShowFinder code, the experiments in §4, and the `T4: AI migration` and `T4: fixes` commits. The instructor may ask you to point to the answer in code or DevTools, or to modify code live without AI assistance. How the defense is graded is described in §8.

Preparation questions:

TypeScript types

- How does an `interface` differ from a `type`?
- What is structural typing? Why does an API object with extra fields satisfy `Show`?
- What is type inference? When is it worthwhile to write types explicitly?
- How does TypeScript handle `null` under `strictNullChecks`? How did you describe fields that can be `null`?
- What is `readonly` used for?

Union types, narrowing, and type guards

- What are union types and discriminated unions? Why is `ViewState` a union rather than a single object with optional fields?
- What mechanisms does TypeScript provide for type narrowing? Give examples from your code.
- What is a type guard (`value is Show`)? What exactly does your `isShow` guarantee, and what does it not guarantee?
- Why use a variable of type `never` in the `default` branch?
- Why does the variable in `catch (err)` have type `unknown` in strict mode? Why is `instanceof TypeError` an unreliable check for network failures?

any, unknown, and type assertions

- How does `unknown` differ from `any`? Why is `any` called "infectious"? Where does `any` come from in code when nobody explicitly wrote it?
- What does the compiler verify in `value as Show`, and what does it ignore? Why is `'abc' as number` an error, while `'abc' as unknown as number` is not?
- What does `!` after an expression promise? How does `element!.focus()` differ from `element?.focus()`?
- How does `document.querySelector<HTMLFormElement>('#search')` differ from `document.querySelector('form')`? Why is the former also a type assertion?
- How do you narrow `event.target` to the element that was actually clicked?
- What is declaration merging? Why is `declare global { interface EventTarget { … } }` also a type assertion, and why does §5 forbid it?
- What does `as const` do and why does §5 allow it? What does `satisfies` check?

Compiler and strict mode

- What does `tsc` do? What remains of types after compilation?
- What does `strict` turn on? Which errors disappeared in E2, and which of those would cause runtime failures?
- Why use `import type`?
- How does `@ts-expect-error` differ from `@ts-ignore`? Why are both forbidden in `src/`?

- How does type checking differ from automated tests? In T1–T4, what is verified by the compiler and what is verified by tests?

Runtime data validation

- Why does the `Show` type not guarantee that the network response is actually a show?
- Where is the validation boundary in ShowFinder? Why is it located in `api.ts` rather than `view.ts`?
- What would the user see if TVmaze changes its response schema — with your validation versus without it?
- How do you validate the entire response schema rather than three properties? What do schema libraries like Zod offer for this?

AI-assisted migration from JavaScript

- How do you migrate a project to TypeScript incrementally? What do `allowJs`, `checkJs`, and the `// @ts-check` comment do?
- Which defects in ShowFinder's JavaScript modules were caught by the compiler, which only by tests, and which did the agent fix on its own? Why didn't the compiler flag `fetch(...).then((response) => response.json())`?
- What in the agent's migration merely masked an error instead of fixing it? How did you detect that?
- Why must the agent not modify `tsconfig.json` and `check/`? What would change if it turned off `strict`?
- How can you constrain an agent in advance: what would you write in an agent rules file (`AGENTS.md`) or configure in linter rules (`no-explicit-any`, `no-non-null-assertion`)?

8. Grading

This lab takes two curriculum slots — Lab 12 and Lab 13 — so it is worth **4 points** across three components, plus **1 bonus point** for timely submission.

Component	Points	How it is graded
Self-check	1	`npm run check` in your repository: "Types: no errors" and all tests pass — 1; no type errors and at least 90% of tests pass (44 of 48) — 0.5; otherwise — 0
Process	1	commits following §5.3 and all report items from §6 present — 1; missing one item — 0.5; missing more — 0
Defense	2	questions about your own code — up to 1; two questions from different groups in §7 — 0.5 each for a complete answer
Bonus point	+1	report email sent by **22.10.2026** inclusive